Data Preparation Guide - Step by Step

Student Dropout Prediction Project

Overview

This guide walks you through CRISP-DM Phase 3: Data Preparation for your student dropout prediction project.

Estimated Time: 6-8 hours **Prerequisite:** Completed Data Understanding (EDA)

PHASE 3: DATA PREPARATION CHECKLIST

✅ Step 1: Setup & Load Data (15 minutes)

- [ ] Import all necessary libraries
- [ ] Load the cleaned dataset from Phase 2
- [ ] Verify data shape and types
- [ ] Create a copy for preprocessing

✅ Step 2: Feature Engineering (1-2 hours)

- [ ] Create semester performance trends
- [ ] Create grade improvement indicators
- [ ] Create academic performance ratios
- [ ] Create interaction features
- [ ] Document all new features

✅ Step 3: Feature Selection (1 hour)

- [ ] Remove highly correlated features ($r > 0.95$)
- [ ] Remove low-variance features
- [ ] Document removed features and rationale
- [ ] Verify remaining feature count

✅ Step 4: Encoding & Transformation (30 minutes)

- [ ] Verify all categorical encodings
- [ ] Create label mappings for interpretation
- [ ] Check for any remaining non-numeric data

✅ Step 5: Train-Test Split (15 minutes)

- [ ] Separate features (X) and target (y)
- [ ] Perform stratified 80/20 split
- [ ] Verify class distribution in both sets
- [ ] Save split indices for reproducibility

✅ Step 6: Handle Class Imbalance (1 hour)

- [ ] Apply SMOTE to training data only
- [ ] Compare distributions before/after
- [ ] Test class weighting as alternative
- [ ] Document chosen approach

✅ Step 7: Algorithm-Specific Preprocessing (1-2 hours)

- [ ] Create StandardScaler pipeline (for SVM, Neural Networks, K-Means)
- [ ] Keep raw data copy (for Decision Tree, Random Forest)
- [ ] Create MinMaxScaler pipeline (for Neural Networks alternative)
- [ ] Test all pipelines

✅ Step 8: Save Preprocessed Data (15 minutes)

- [ ] Save all preprocessed datasets
- [ ] Save preprocessing pipelines (.pkl files)
- [ ] Save feature lists
- [ ] Create data dictionary

✅ Step 9: Validation & Quality Checks (30 minutes)

- [ ] Check for data leakage
- [ ] Verify no missing values introduced
- [ ] Check feature distributions
- [ ] Validate data shapes

✅ Step 10: Documentation (30 minutes)

- [ ] Update project report Section 3
- [ ] Document all decisions and rationale
- [ ] Create preprocessing summary table
- [ ] Save all code with comments

DETAILED STEP-BY-STEP INSTRUCTIONS

Step 1: Setup & Load Data

Code Block 1.1: Import Libraries

```python
# Core libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.feature_selection import VarianceThreshold

# Handling imbalance
from imblearn.over_sampling import SMOTE

# Save models
import pickle

# Utilities
import warnings
warnings.filterwarnings('ignore')

# Set display options
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)

# Set random seed for reproducibility
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("✅ All libraries imported successfully!")
```

Code Block 1.2: Load Data

```python
```

```python
# Load your dataset
df = pd.read_csv('data.csv')

print(f"Dataset shape: {df.shape}")
print(f"Features: {df.shape[1] - 1}")  # -1 for target
print(f"Records: {df.shape[0]}")
print(f"\nTarget distribution:\n{df['Target'].value_counts()}")
print(f"\nTarget percentages:\n{df['Target'].value_counts(normalize=True) * 100}")

# Create a working copy
df_prep = df.copy()
print("\n✅ Data loaded successfully!")
```

Code Block 1.3: Initial Data Check

```python
# Check data types
print("Data types:")
print(df_prep.dtypes.value_counts())

# Check for missing values
print(f"\nMissing values: {df_prep.isnull().sum().sum()}")

# Check memory usage
print(f"\nMemory usage: {df_prep.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
```

✅ **Checkpoint:** Verify you have 4,424 rows and 37 columns with no missing values.

Step 2: Feature Engineering

Code Block 2.1: Semester Performance Trends

```python
```

```python
# Calculate grade improvement from 1st to 2nd semester
df_prep['Grade_Improvement'] = (
    df_prep['Curricular units 2nd sem (grade)'] -
    df_prep['Curricular units 1st sem (grade)']
)

# Calculate approval rate improvement
df_prep['Approval_Rate_1st_Sem'] = (
    df_prep['Curricular units 1st sem (approved)'] /
    df_prep['Curricular units 1st sem (enrolled)'].replace(0, 1)  # Avoid division by zero
)

df_prep['Approval_Rate_2nd_Sem'] = (
    df_prep['Curricular units 2nd sem (approved)'] /
    df_prep['Curricular units 2nd sem (enrolled)'].replace(0, 1)
)

df_prep['Approval_Rate_Improvement'] = (
    df_prep['Approval_Rate_2nd_Sem'] - df_prep['Approval_Rate_1st_Sem']
)

print("✅ Semester trend features created!")
print(f"Grade_Improvement range: {df_prep['Grade_Improvement'].min():.2f} to {df_prep['Grade_Improvement'].max():.2f}
print(f"Approval_Rate_1st_Sem range: {df_prep['Approval_Rate_1st_Sem'].min():.2f} to {df_prep['Approval_Rate_1st_Sem
```

Code Block 2.2: Academic Performance Indicators

```python
```

```python
# Total units approved across both semesters
df_prep['Total_Units_Approved'] = (
    df_prep['Curricular units 1st sem (approved)'] +
    df_prep['Curricular units 2nd sem (approved)']
)

# Average grade across both semesters
df_prep['Average_Grade'] = (
    df_prep['Curricular units 1st sem (grade)'] +
    df_prep['Curricular units 2nd sem (grade)']
) / 2

# Academic struggle indicator (units enrolled but not evaluated)
df_prep['Units_Without_Eval'] = (
    df_prep['Curricular units 1st sem (without evaluations)'] +
    df_prep['Curricular units 2nd sem (without evaluations)']
)

# Consistency indicator (lower = more consistent)
df_prep['Grade_Consistency'] = abs(
    df_prep['Curricular units 1st sem (grade)'] -
    df_prep['Curricular units 2nd sem (grade)']
)

print("✅ Academic performance indicators created!")
```

Code Block 2.3: Socio-Economic Composite Features

```python
# Parental education level (average of mother and father)
df_prep['Parental_Education_Avg'] = (
    df_prep["Mother's qualification"] +
    df_prep["Father's qualification"]
) / 2

# Financial support indicator (scholarship + fees up to date - debtor)
df_prep['Financial_Support_Score'] = (
    df_prep['Scholarship holder'] +
    df_prep['Tuition fees up to date'] -
    df_prep['Debtor']
)

print("✅ Socio-economic composite features created!")
```

Code Block 2.4: Interaction Features

```python
# Age × Previous qualification (mature students with lower qualifications at higher risk)
df_prep['Age_X_PrevQual'] = (
    df_prep['Age at enrollment'] *
    df_prep['Previous qualification (grade)']
)

# Admission grade × 1st semester performance
df_prep['Admission_X_1stSemGrade'] = (
    df_prep['Admission grade'] *
    df_prep['Curricular units 1st sem (grade)']
)

print("✅ Interaction features created!")
```

Code Block 2.5: Display New Features

```python
```

```python
# List all new features
new_features = [
    'Grade_Improvement',
    'Approval_Rate_1st_Sem',
    'Approval_Rate_2nd_Sem',
    'Approval_Rate_Improvement',
    'Total_Units_Approved',
    'Average_Grade',
    'Units_Without_Eval',
    'Grade_Consistency',
    'Parental_Education_Avg',
    'Financial_Support_Score',
    'Age_X_PrevQual',
    'Admission_X_1stSemGrade'
]

print(f"Total new features created: {len(new_features)}")
print(f"New dataset shape: {df_prep.shape}")
print(f"\nNew features:\n")
for i, feat in enumerate(new_features, 1):
    print(f"{i}. {feat}")

# Display sample statistics
print(f"\nSample statistics for new features:")
print(df_prep[new_features].describe())
```

✅ **Checkpoint:** You should now have 49 columns (37 original + 12 new features).

Step 3: Feature Selection

Code Block 3.1: Calculate Correlation Matrix

```python
```

```python
# Calculate correlations (exclude target)
feature_cols = [col for col in df_prep.columns if col != 'Target']
corr_matrix = df_prep[feature_cols].corr()

# Find highly correlated pairs (r > 0.95)
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.95:
            high_corr_pairs.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                corr_matrix.iloc[i, j]
            ))

print(f"Found {len(high_corr_pairs)} highly correlated pairs (r > 0.95):")
for feat1, feat2, corr_val in high_corr_pairs:
    print(f"  {feat1} ↔ {feat2}: r = {corr_val:.3f}")
```

Code Block 3.2: Remove Redundant Features

```python
# Based on your EDA, you identified these high correlations:
# 1. Curricular units 1st sem (credited) ↔ 2nd sem (credited): r = 0.945
# 2. Curricular units 1st sem (enrolled) ↔ 2nd sem (enrolled): r = 0.943
# 3. Mother's occupation ↔ Father's occupation: r = 0.910
# 4. Curricular units 1st sem (approved) ↔ 2nd sem (approved): r = 0.904

# Strategy: Keep 1st semester features, remove 2nd semester duplicates
# Rationale: For early prediction (after 1st semester)

features_to_remove = [
    'Curricular units 2nd sem (credited)',  # Keep 1st sem version
    'Curricular units 2nd sem (enrolled)',  # Keep 1st sem version
    "Father's occupation",                  # Keep Mother's occupation (or create average)
]

print(f"Removing {len(features_to_remove)} redundant features:")
for feat in features_to_remove:
    print(f"  - {feat}")

df_prep = df_prep.drop(columns=features_to_remove)
print(f"\n✅ New shape after removing redundant features: {df_prep.shape}")
```

Code Block 3.3: Remove Low-Variance Features

```python
# Identify low-variance features (exclude target)
feature_cols = [col for col in df_prep.columns if col != 'Target']

# Calculate variance for each feature
variances = df_prep[feature_cols].var()
low_var_features = variances[variances < 0.01].index.tolist()

print(f"Low-variance features (variance < 0.01): {len(low_var_features)}")
if low_var_features:
    print("Features to consider removing:")
    for feat in low_var_features:
        print(f"  - {feat}: variance = {variances[feat]:.6f}")

    # Remove if any (typically binary features with very skewed distribution)
    # Be careful - don't remove important features!
    # Review each one before removing

    # Example: If 'Educational special needs' has very low variance (99% are 0)
    # you might choose to keep it if it's theoretically important

    print("\n⚠️  REVIEW REQUIRED: Manually decide which low-variance features to remove")
else:
    print("No extremely low-variance features found.")
```

Code Block 3.4: Document Feature Selection

```python
# Final feature count
feature_cols = [col for col in df_prep.columns if col != 'Target']
print(f"\n📊 FEATURE SELECTION SUMMARY:")
print(f"  Original features: 37")
print(f"  New engineered features: {len(new_features)}")
print(f"  Removed redundant features: {len(features_to_remove)}")
print(f"  Final feature count: {len(feature_cols)}")
print(f"  Final dataset shape: {df_prep.shape}")
```

✅ **Checkpoint:** You should have approximately 46 features remaining.

Step 4: Encoding & Transformation

Code Block 4.1: Verify Encodings

```python
# Check data types - all should be numeric except Target
print("Data types after feature engineering:")
print(df_prep.dtypes.value_counts())

# Target should be the only non-numeric column
non_numeric = df_prep.select_dtypes(include=['object']).columns.tolist()
print(f"\nNon-numeric columns: {non_numeric}")

if non_numeric == ['Target']:
    print("✅ All features are numeric - ready for modeling!")
else:
    print("⚠️ Warning: Found unexpected non-numeric columns")
```

Code Block 4.2: Create Label Mappings

```python
# Create mapping for target variable for later interpretation
target_mapping = {
    'Dropout': 0,
    'Enrolled': 1,
    'Graduate': 2
}

# Save reverse mapping for interpretation
reverse_target_mapping = {v: k for k, v in target_mapping.items()}

# Encode target variable
df_prep['Target_Encoded'] = df_prep['Target'].map(target_mapping)

print("Target encoding:")
print(df_prep[['Target', 'Target_Encoded']].drop_duplicates().sort_values('Target_Encoded'))

# Verify no missing values in encoding
print(f"\nMissing values in Target_Encoded: {df_prep['Target_Encoded'].isnull().sum()}")
print("✅ Target variable encoded successfully!")
```

Code Block 4.3: Save Mappings

```python
```

```python
# Save mappings for future use
mappings = {
    'target_mapping': target_mapping,
    'reverse_target_mapping': reverse_target_mapping,
    'feature_names': [col for col in df_prep.columns if col not in ['Target', 'Target_Encoded']]
}

with open('label_mappings.pkl', 'wb') as f:
    pickle.dump(mappings, f)

print("✅ Label mappings saved to 'label_mappings.pkl'")
```

✅ **Checkpoint:** Target should be encoded as 0 (Dropout), 1 (Enrolled), 2 (Graduate).

Step 5: Train-Test Split

Code Block 5.1: Separate Features and Target

```python
# Separate features (X) and target (y)
X = df_prep.drop(columns=['Target', 'Target_Encoded'])
y = df_prep['Target_Encoded']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"\nFeature list ({len(X.columns)} features):")
for i, col in enumerate(X.columns, 1):
    print(f"{i}. {col}")
```

Code Block 5.2: Stratified Train-Test Split

```python
```

```python
# 80/20 split with stratification to maintain class distribution
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y  # Maintain class distribution
)

print(f"Training set: {X_train.shape[0]} samples ({X_train.shape[0]/len(X)*100:.1f}%)")
print(f"Test set: {X_test.shape[0]} samples ({X_test.shape[0]/len(X)*100:.1f}%)")

# Verify class distribution
print("\n📊 Class distribution:")
print("\nOriginal:")
print(y.value_counts(normalize=True).sort_index())
print("\nTraining set:")
print(y_train.value_counts(normalize=True).sort_index())
print("\nTest set:")
print(y_test.value_counts(normalize=True).sort_index())

print("\n✅ Stratified split successful - distributions match!")
```

Code Block 5.3: Save Split Indices

```python
# Save indices for reproducibility
split_indices = {
    'train_indices': X_train.index.tolist(),
    'test_indices': X_test.index.tolist()
}

with open('train_test_split_indices.pkl', 'wb') as f:
    pickle.dump(split_indices, f)

print("✅ Train-test split indices saved!")
```

✅ **Checkpoint:** Verify class distributions are nearly identical across train/test sets.

Step 6: Handle Class Imbalance

Code Block 6.1: Apply SMOTE (Training Data Only!)

```python
# IMPORTANT: Only apply SMOTE to training data to avoid data leakage!

print("Before SMOTE:")
print(f"Training set size: {len(y_train)}")
print(f"Class distribution:\n{y_train.value_counts()}")
print(f"Class percentages:\n{y_train.value_counts(normalize=True) * 100}")

# Apply SMOTE
smote = SMOTE(random_state=RANDOM_STATE, k_neighbors=5)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)

print("\n\nAfter SMOTE:")
print(f"Training set size: {len(y_train_balanced)}")
print(f"Class distribution:\n{y_train_balanced.value_counts()}")
print(f"Class percentages:\n{y_train_balanced.value_counts(normalize=True) * 100}")

print("\n✅ SMOTE applied successfully!")
print(f"⚠️ Note: Test set unchanged (no SMOTE applied)")
```

Code Block 6.2: Visualize Before/After SMOTE

```python
```

```python
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Before SMOTE
y_train.value_counts().sort_index().plot(kind='bar', ax=axes[0], color=['#e74c3c', '#f39c12', '#27ae60'])
axes[0].set_title('Training Set - Before SMOTE', fontsize=14, fontweight='bold')
axes[0].set_xlabel('Class', fontsize=12)
axes[0].set_ylabel('Count', fontsize=12)
axes[0].set_xticklabels(['Dropout', 'Enrolled', 'Graduate'], rotation=0)
axes[0].grid(axis='y', alpha=0.3)

# After SMOTE
y_train_balanced.value_counts().sort_index().plot(kind='bar', ax=axes[1], color=['#e74c3c', '#f39c12', '#27ae60'])
axes[1].set_title('Training Set - After SMOTE', fontsize=14, fontweight='bold')
axes[1].set_xlabel('Class', fontsize=12)
axes[1].set_ylabel('Count', fontsize=12)
axes[1].set_xticklabels(['Dropout', 'Enrolled', 'Graduate'], rotation=0)
axes[1].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('smote_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

print("✅ Visualization saved as 'smote_comparison.png'")
```

Code Block 6.3: Prepare Both Versions

```python
```

```python
# Keep both balanced and unbalanced versions for experimentation
datasets = {
    'unbalanced': {
        'X_train': X_train,
        'y_train': y_train,
        'X_test': X_test,
        'y_test': y_test
    },
    'balanced': {
        'X_train': X_train_balanced,
        'y_train': y_train_balanced,
        'X_test': X_test,  # Test set never changes
        'y_test': y_test
    }
}

print("✅ Both dataset versions prepared!")
print("\nYou can test models with both to see which performs better.")
```

✅ **Checkpoint:** Training set should be balanced (equal class sizes), test set unchanged.

Step 7: Algorithm-Specific Preprocessing

Code Block 7.1: StandardScaler Pipeline (for SVM, Neural Networks, K-Means)

```python
```

```python
# Create StandardScaler for distance-based algorithms
scaler_standard = StandardScaler()

# Fit on training data only (prevent data leakage)
scaler_standard.fit(X_train)

# Transform both train and test
X_train_scaled = scaler_standard.transform(X_train)
X_test_scaled = scaler_standard.transform(X_test)

# Convert back to DataFrame (optional, for easier handling)
X_train_scaled_df = pd.DataFrame(
    X_train_scaled,
    columns=X_train.columns,
    index=X_train.index
)

X_test_scaled_df = pd.DataFrame(
    X_test_scaled,
    columns=X_test.columns,
    index=X_test.index
)

print("✅ StandardScaler applied!")
print(f"Training set mean (should be ~0): {X_train_scaled_df.mean().mean():.6f}")
print(f"Training set std (should be ~1): {X_train_scaled_df.std().mean():.3f}")

# Save the scaler
with open('standard_scaler.pkl', 'wb') as f:
    pickle.dump(scaler_standard, f)
print("✅ StandardScaler saved to 'standard_scaler.pkl'")
```

Code Block 7.2: StandardScaler for Balanced Data

```python
# Apply same scaling to balanced training data
X_train_balanced_scaled = scaler_standard.transform(X_train_balanced)

X_train_balanced_scaled_df = pd.DataFrame(
    X_train_balanced_scaled,
    columns=X_train.columns  # Use original column names
)

print("✅ StandardScaler applied to balanced training data!")
```

Code Block 7.3: MinMaxScaler Pipeline (Alternative for Neural Networks)

```python
# Create MinMaxScaler (scales to [0, 1])
scaler_minmax = MinMaxScaler()

# Fit on training data only
scaler_minmax.fit(X_train)

# Transform
X_train_minmax = scaler_minmax.transform(X_train)
X_test_minmax = scaler_minmax.transform(X_test)

X_train_minmax_df = pd.DataFrame(
    X_train_minmax,
    columns=X_train.columns,
    index=X_train.index
)

X_test_minmax_df = pd.DataFrame(
    X_test_minmax,
    columns=X_test.columns,
    index=X_test.index
)

print("✅ MinMaxScaler applied!")
print(f"Training set min: {X_train_minmax_df.min().min():.6f}")
print(f"Training set max: {X_train_minmax_df.max().max():.6f}")

# Save the scaler
with open('minmax_scaler.pkl', 'wb') as f:
    pickle.dump(scaler_minmax, f)
print("✅ MinMaxScaler saved to 'minmax_scaler.pkl'")
```

Code Block 7.4: Prepare All Dataset Versions

```python
```

```python
# Summary of all prepared datasets
all_datasets = {

    # For Decision Trees, Random Forest (no scaling needed)
    'tree_based_unbalanced': {
        'X_train': X_train,
        'y_train': y_train,
        'X_test': X_test,
        'y_test': y_test
    },

    'tree_based_balanced': {
        'X_train': X_train_balanced,
        'y_train': y_train_balanced,
        'X_test': X_test,
        'y_test': y_test
    },

    # For SVM, K-Means (StandardScaler)
    'standard_scaled_unbalanced': {
        'X_train': X_train_scaled_df,
        'y_train': y_train,
        'X_test': X_test_scaled_df,
        'y_test': y_test
    },

    'standard_scaled_balanced': {
        'X_train': X_train_balanced_scaled_df,
        'y_train': y_train_balanced,
        'X_test': X_test_scaled_df,
        'y_test': y_test
    },

    # For Neural Networks (MinMaxScaler)
    'minmax_scaled_unbalanced': {
        'X_train': X_train_minmax_df,
        'y_train': y_train,
        'X_test': X_test_minmax_df,
        'y_test': y_test
    }
}

print("✅ All dataset versions prepared!")
print(f"\nTotal dataset versions: {len(all_datasets)}")
```

```
for name in all_datasets.keys():
    print(f"  - {name}")
```

✅ **Checkpoint:** You should have 5 dataset versions ready for different algorithms.

Step 8: Save Preprocessed Data

Code Block 8.1: Save Main Datasets

```python
# Save unbalanced versions (CSV)
X_train.to_csv('X_train_unbalanced.csv', index=False)
y_train.to_csv('y_train_unbalanced.csv', index=False, header=True)
X_test.to_csv('X_test.csv', index=False)
y_test.to_csv('y_test.csv', index=False, header=True)

# Save balanced versions (CSV)
X_train_balanced.to_csv('X_train_balanced.csv', index=False)
y_train_balanced.to_csv('y_train_balanced.csv', index=False, header=True)

# Save scaled versions (NumPy arrays for efficiency)
np.save('X_train_scaled.npy', X_train_scaled)
np.save('X_test_scaled.npy', X_test_scaled)
np.save('X_train_balanced_scaled.npy', X_train_balanced_scaled)

print("✅ All datasets saved!")
```

Code Block 8.2: Save All Preprocessing Objects

```python
```

```python
# Bundle all preprocessing objects
preprocessing_objects = {
    'standard_scaler': scaler_standard,
    'minmax_scaler': scaler_minmax,
    'smote': smote,
    'feature_names': X_train.columns.tolist(),
    'target_mapping': target_mapping,
    'reverse_target_mapping': reverse_target_mapping,
    'new_features': new_features,
    'removed_features': features_to_remove
}

with open('preprocessing_pipeline.pkl', 'wb') as f:
    pickle.dump(preprocessing_objects, f)

print("✅ Preprocessing pipeline saved to 'preprocessing_pipeline.pkl'")
```

Code Block 8.3: Create Data Dictionary

```python
```

```python
# Create comprehensive data dictionary
data_dictionary = {
    'dataset_info': {
        'original_shape': (4424, 37),
        'final_shape': df_prep.shape,
        'train_size': len(X_train),
        'test_size': len(X_test),
        'train_balanced_size': len(X_train_balanced)
    },
    'features': {
        'original_count': 37,
        'engineered_count': len(new_features),
        'removed_count': len(features_to_remove),
        'final_count': len(X_train.columns)
    },
    'feature_list': X_train.columns.tolist(),
    'new_features': new_features,
    'removed_features': features_to_remove,
    'target_classes': ['Dropout', 'Enrolled', 'Graduate'],
    'target_distribution': {
        'original': y.value_counts().to_dict(),
        'train': y_train.value_counts().to_dict(),
        'test': y_test.value_counts().to_dict(),
        'train_balanced': y_train_balanced.value_counts().to_dict()
    }
}

with open('data_dictionary.pkl', 'wb') as f:
    pickle.dump(data_dictionary, f)

print("✅ Data dictionary saved!")
```

✅ **Checkpoint:** All files should be saved in your working directory.

Step 9: Validation & Quality Checks

Code Block 9.1: Check for Data Leakage

```python
```

```python
# Verify no overlap between train and test indices
train_indices_set = set(X_train.index)
test_indices_set = set(X_test.index)

overlap = train_indices_set.intersection(test_indices_set)

if len(overlap) == 0:
    print("✅ No data leakage - train and test sets are completely separate!")
else:
    print(f"⚠️ WARNING: Found {len(overlap)} overlapping indices!")
```

Code Block 9.2: Verify No Missing Values

```python
```

```python
# Check all datasets for missing values
datasets_to_check = [
    ('X_train', X_train),
    ('X_test', X_test),
    ('X_train_balanced', X_train_balanced),
    ('y_train', y_train),
    ('y_test', y_test),
    ('y_train_balanced', y_train_balanced)
]

print("Missing value check:")
all_clean = True
for name, data in datasets_to_check:
    missing = data.isnull().sum().sum() if hasattr(data, 'isnull') else 0
    print(f"  {name}: {missing} missing values")
    if missing > 0:
        all_clean = False

if all_clean:
    print("\n✅ No missing values in any dataset!")
else:
    print("\n⚠️ WARNING: Missing values detected!")
```

Code Block 9.3: Check Feature Distributions

```python
```

```python
# Compare feature distributions between train and test
print("Feature distribution comparison (sample of 5 features):")
sample_features = X_train.columns[:5]

for feat in sample_features:
    train_mean = X_train[feat].mean()
    test_mean = X_test[feat].mean()
    diff_pct = abs(train_mean - test_mean) / train_mean * 100 if train_mean != 0 else 0

    print(f"\n{feat}:")
    print(f"  Train mean: {train_mean:.3f}")
    print(f"  Test mean: {test_mean:.3f}")
    print(f"  Difference: {diff_pct:.2f}%")

print("\n✅ Distributions should be similar (differences < 5% typically)")
```

Code Block 9.4: Validate Shapes

```python
```

```python
# Final shape validation
print("📊 FINAL SHAPE VALIDATION:")
print(f"\nTraining data:")
print(f"  X_train: {X_train.shape}")
print(f"  y_train: {y_train.shape}")
print(f"  X_train_balanced: {X_train_balanced.shape}")
print(f"  y_train_balanced: {y_train_balanced.shape}")

print(f"\nTest data:")
print(f"  X_test: {X_test.shape}")
print(f"  y_test: {y_test.shape}")

# Verify consistency
assert X_train.shape[1] == X_test.shape[1], "Feature count mismatch!"
assert X_train.shape[0] == y_train.shape[0], "Train X-y size mismatch!"
assert X_test.shape[0] == y_test.shape[0], "Test X-y size mismatch!"
assert X_train_balanced.shape[1] == X_train.shape[1], "Balanced feature count mismatch!"

print("\n✅ All shapes validated successfully!")
```

✅ **Checkpoint:** All checks should pass with no errors.

Step 10: Documentation

Code Block 10.1: Create Preprocessing Summary Table

python

```python
# Create summary table
summary_data = {
    'Step': [
        'Original Data',
        'Feature Engineering',
        'Feature Selection',
        'Train-Test Split',
        'SMOTE (Training Only)',
        'StandardScaler',
        'MinMaxScaler'
    ],
    'Action': [
        'Loaded UCI dataset',
        f'Created {len(new_features)} new features',
        f'Removed {len(features_to_remove)} redundant features',
        '80/20 stratified split',
        'Balanced training set classes',
        'Scaled features (mean=0, std=1)',
        'Scaled features (range=[0,1])'
    ],
    'Result': [
        f'{df.shape[0]} records × {df.shape[1]} features',
        f'{df_prep.shape[0]} records × {df_prep.shape[1]} features',
        f'{df_prep.shape[0]} records × {len(X_train.columns)+1} features',
        f'Train: {len(X_train)} | Test: {len(X_test)}',
        f'Train: {len(X_train_balanced)} (balanced)',
        'Ready for SVM, Neural Networks, K-Means',
        'Alternative for Neural Networks'
    ]
}

summary_df = pd.DataFrame(summary_data)
print("PREPROCESSING SUMMARY TABLE:")
print(summary_df.to_string(index=False))

# Save summary
summary_df.to_csv('preprocessing_summary.csv', index=False)
print("\n✅ Summary table saved to 'preprocessing_summary.csv'")
```

Code Block 10.2: Generate Final Report Section

```python
```

```
report_text = f"""
==================================================
DATA PREPARATION PHASE - SUMMARY REPORT
==================================================

Date: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M')}
Phase: CRISP-DM Phase 3 - Data Preparation

1. FEATURE ENGINEERING
  - Original features: 37
  - New features created: {len(new_features)}
  - Total before selection: {37 + len(new_features)}

  New Features:
  {chr(10).join([f'  • {feat}' for feat in new_features])}

2. FEATURE SELECTION
  - Features removed: {len(features_to_remove)}
  - Final feature count: {len(X_train.columns)}

  Removed Features (high correlation):
  {chr(10).join([f'  • {feat}' for feat in features_to_remove])}

3. DATA SPLITTING
  - Training set: {len(X_train)} samples ({len(X_train)/len(df)*100:.1f}%)
  - Test set: {len(X_test)} samples ({len(X_test)/len(df)*100:.1f}%)
  - Split method: Stratified 80/20

4. CLASS IMBALANCE HANDLING
  - Method: SMOTE (Synthetic Minority Over-sampling)
  - Applied to: Training set only
  - Before SMOTE: {len(y_train)} samples
  - After SMOTE: {len(y_train_balanced)} samples

5. PREPROCESSING PIPELINES CREATED
  - StandardScaler: For SVM, Neural Networks, K-Means
  - MinMaxScaler: Alternative for Neural Networks
  - Raw data: For Decision Trees, Random Forest

6. DATASETS PREPARED
  Total dataset versions: 5
  - tree_based_unbalanced
  - tree_based_balanced
  - standard_scaled_unbalanced
  - standard_scaled_balanced
  - minmax_scaled_unbalanced
```

```
7. QUALITY CHECKS
  ✅ No data leakage detected
  ✅ No missing values introduced
  ✅ Feature distributions consistent
  ✅ All shapes validated

8. FILES SAVED
  - X_train_unbalanced.csv
  - X_train_balanced.csv
  - X_test.csv
  - y_train_unbalanced.csv
  - y_train_balanced.csv
  - y_test.csv
  - standard_scaler.pkl
  - minmax_scaler.pkl
  - preprocessing_pipeline.pkl
  - label_mappings.pkl
  - data_dictionary.pkl
  - preprocessing_summary.csv

==================================================
STATUS: DATA PREPARATION COMPLETE ✅
READY FOR: Phase 4 - Modeling
==================================================
"""

print(report_text)

# Save report
with open('data_preparation_report.txt', 'w') as f:
    f.write(report_text)

print("✅ Full report saved to 'data_preparation_report.txt'")
```

- ☐ ✅ All feature engineering completed
- ☐ ✅ Redundant features removed
- ☐ ✅ Train-test split performed (80/20, stratified)
- ☐ ✅ SMOTE applied to training data only
- ☐ ✅ StandardScaler and MinMaxScaler pipelines created

- ☑ All datasets saved (CSV and .npy files)
- ☑ All preprocessing objects saved (.pkl files)
- ☑ No data leakage detected
- ☑ No missing values introduced
- ☑ All validation checks passed
- ☑ Documentation completed

FINAL CHECKLIST

Before moving to Phase 4 (Modeling), verify:

NEXT STEPS

You are now ready to proceed to **Phase 4: Modeling**!

Phase 4 will include:

1. Clustering (K-Means, DBSCAN)
2. Classification (Decision Tree, Random Forest, SVM, Neural Network, Logistic Regression)
3. Hyperparameter tuning

4. Model comparison and selection

Estimated time for Phase 4: 10-12 hours

TIPS FOR SUCCESS

1. **Save your work frequently** - Use version control (Git)
2. **Comment your code** - Future you will thank present you
3. **Test incrementally** - Don't wait until the end to run code
4. **Keep backups** - Save important datasets in multiple locations
5. **Document decisions** - Write down WHY you made each choice

---

Good luck with your data preparation! 🚀